

Polling vs Webhooks vs WebSockets

Chapter 2 — WebHooks (Zero → Hero)

1. Why Polling Wasn't Enough

Let's continue the pizza example.

With Polling, your app keeps asking:

```
Is my order ready?
```

```
Is my order ready?
```

```
Is my order ready?
```

Imagine

10 million users.

Each asks every

```
5 seconds
```

Server receives millions of useless requests.

Most responses are

```
{  
  "status": "Preparing"  
}
```

Nothing changed.

Still bandwidth was consumed.

CPU was used.

Database was queried.

Money was spent.

Companies hate this.

2. The Big Idea Behind Webhooks

Instead of asking repeatedly,

what if the server says

"Don't ask me again.

I'll tell you when something changes."

That's exactly a Webhook.

Definition

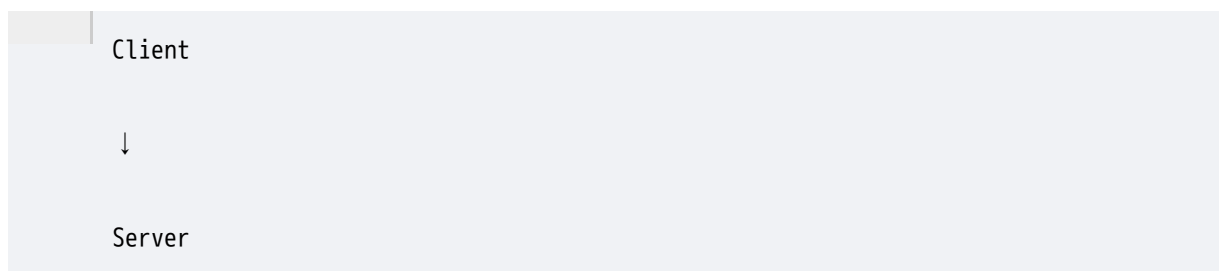
A Webhook is an HTTP callback.

When an event occurs,

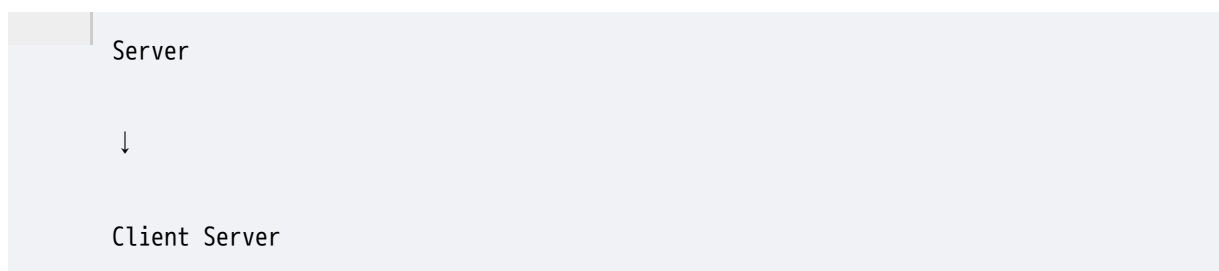
the server automatically sends an HTTP request to another server.

Notice something.

Polling



Webhook



Communication direction changed.

3. Important Clarification

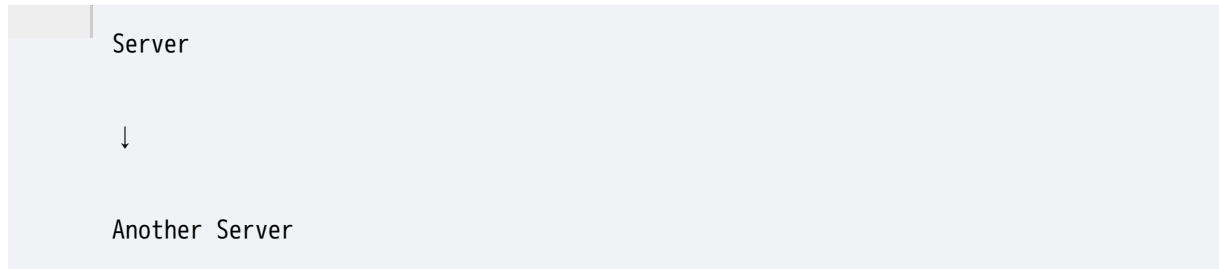
Many beginners think

Webhook means

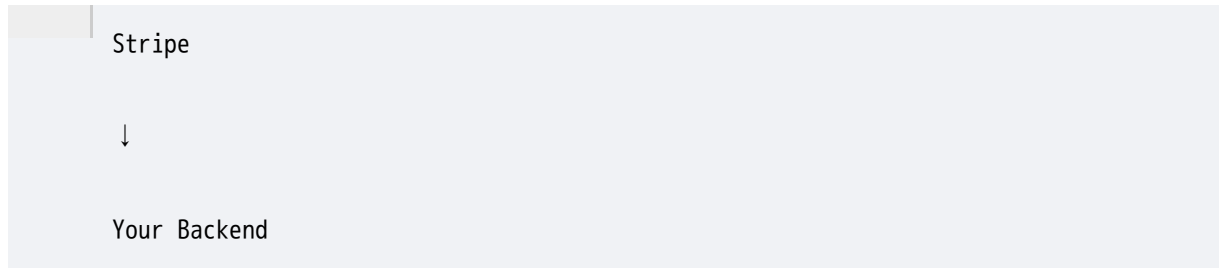
Server → Browser

Wrong.

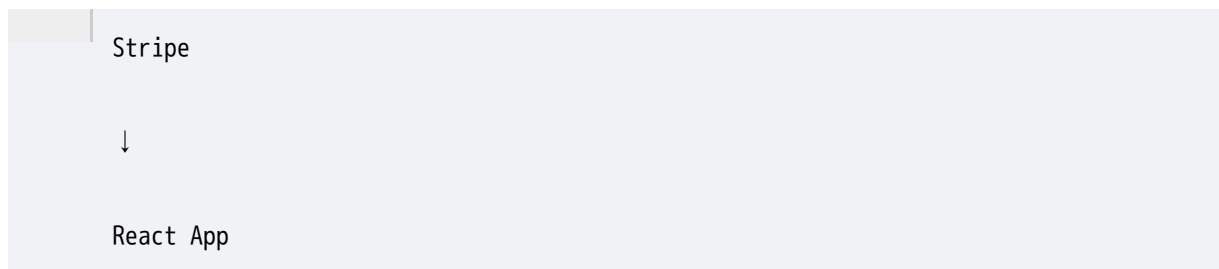
Webhook almost always means



Example



NOT



Why?

Because

Your browser cannot expose a permanent public endpoint.

Your backend can.

This is one of the most common interview traps.

4. Real Life Example

Suppose you own an online shopping website.

Customer pays using Stripe.

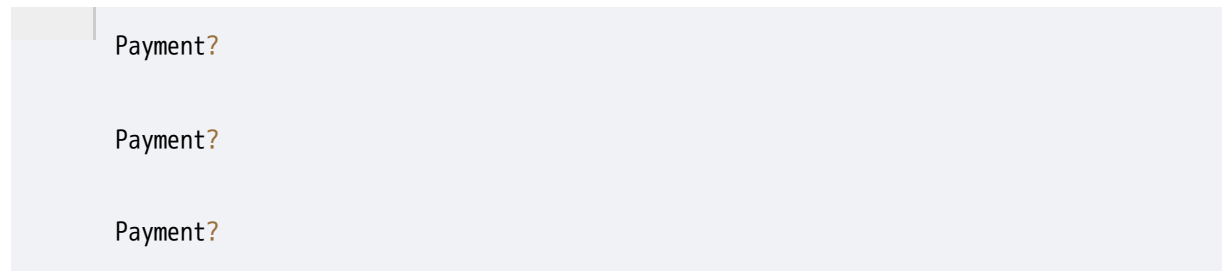
Question

How will your server know

Payment succeeded?

Option 1

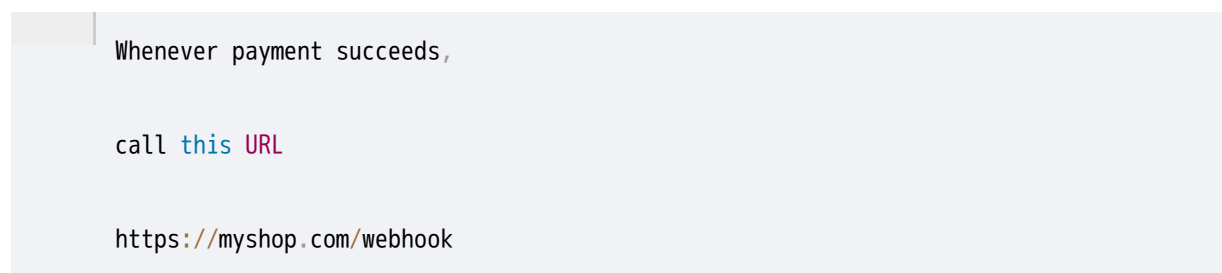
Poll Stripe every second.



Terrible.

Option 2

Tell Stripe

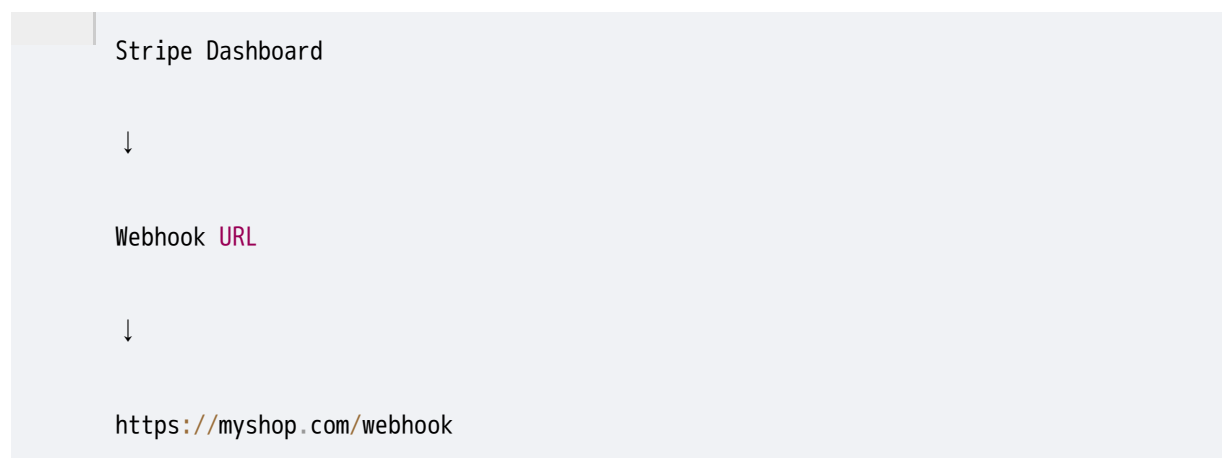


Perfect.

5. Complete Flow

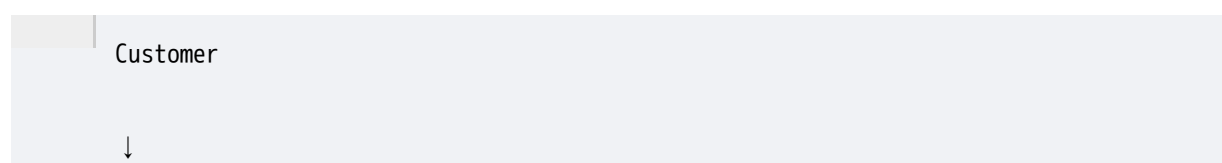
Step 1

Merchant registers webhook URL.



Step 2

Customer pays.



Stripe

Step 3

Payment succeeds.

Stripe generates an event.

```
payment_intent.succeeded
```

Step 4

Stripe sends HTTP POST.

```
POST /webhook

{
  "event": "payment_success",
  "paymentId": "abc123"
}
```

Step 5

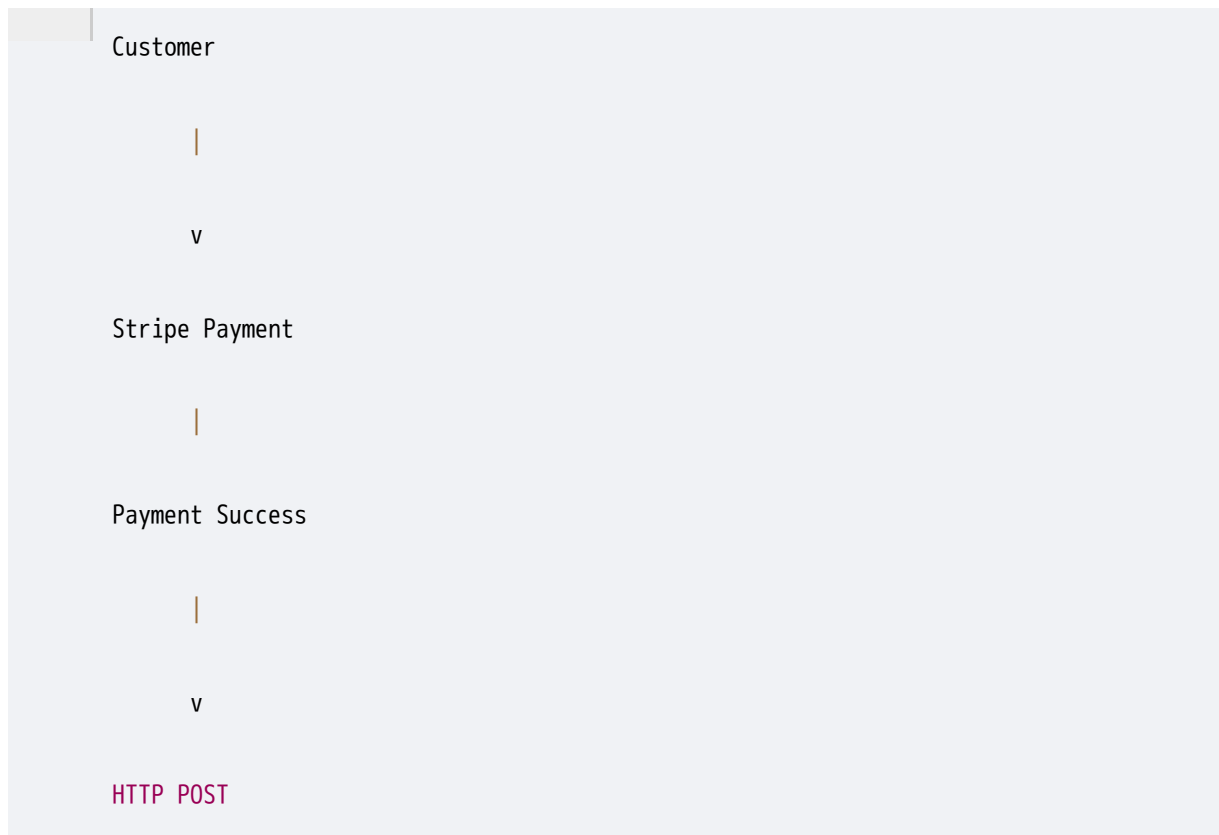
Your backend receives it.

```
Receive Request
↓
Verify Signature
↓
Update Database
↓
Return HTTP 200
```

Done.

No polling.

6. Visual Flow



Notice

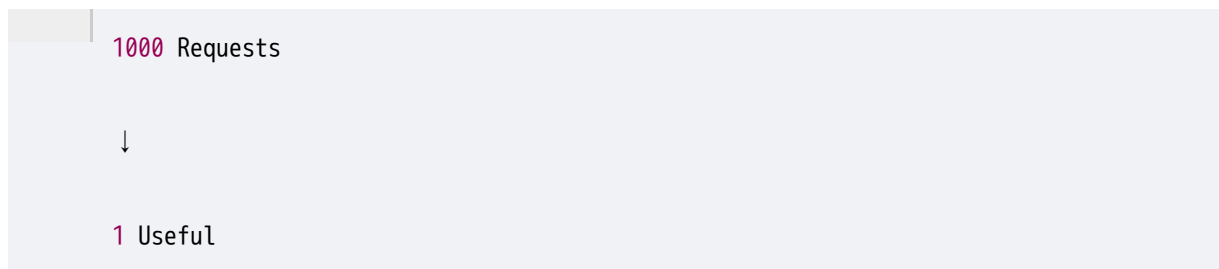
Stripe became the client.

Your backend became the server.

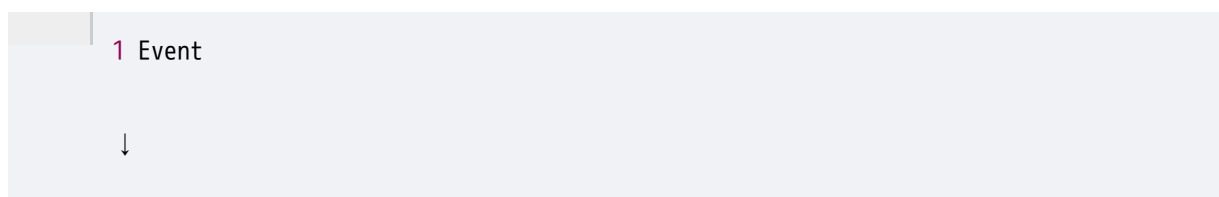
Very important.

7. Why Webhooks Are Efficient

Polling



Webhook



1 Request

Only when required.

Huge bandwidth savings.

8. Real Companies Using Webhooks

STRIPE

Payment Success

Refund

Subscription Renewal

Charge Failed

GITHUB

 git push

Immediately

GitHub sends

 push event

to Jenkins,

GitHub Actions,

Azure DevOps,

or your own backend.

RAZORPAY

Payment Completed

↓

Webhook

↓

Update Order

TWILIO

SMS Delivered

↓

Webhook

↓

Update Dashboard

SHOPIFY

New Order

↓

Webhook

↓

Inventory System

9. What Does a Webhook Look Like?

Typical HTTP request

```
POST /webhook
```

```
Content-Type: application/json
```

```
{  
  "event": "payment_success",  
  
  "paymentId": "12345",  
  
  "amount": 500  
}
```

Your server

```
HTTP 200 OK
```

That's it.

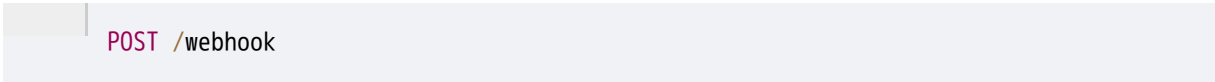
Nothing magical.

Just HTTP.

10. What Happens if Your Server is Down?

Suppose

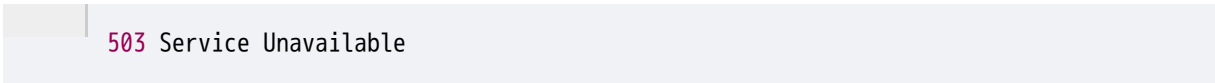
Stripe sends



POST /webhook

But

Your server crashed.



503 Service Unavailable

Now what?

Did the payment disappear?

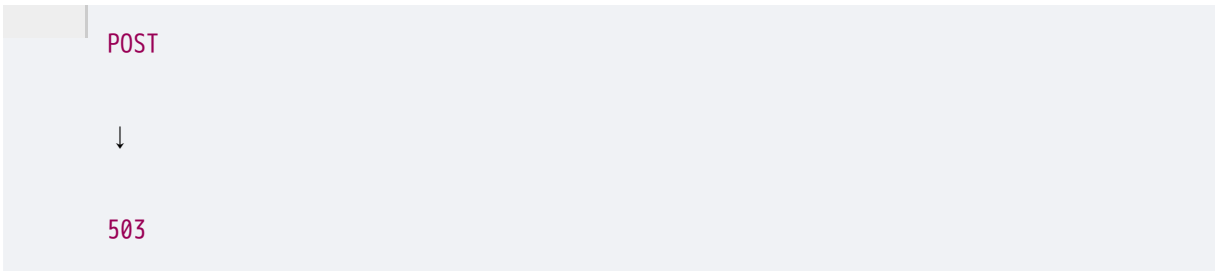
No.

This is where retries come in.

11. Retry Mechanism

Professional webhook providers never assume delivery succeeded.

Flow

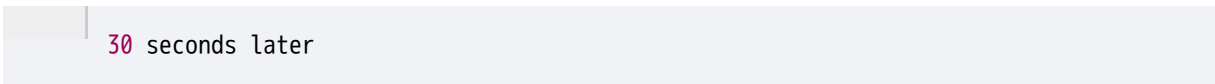


POST

↓

503

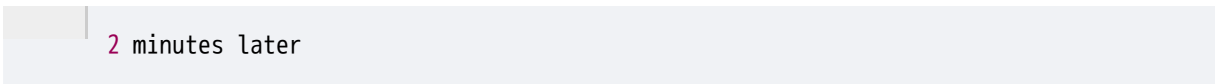
Retry



30 seconds later

Still failed

Retry again



2 minutes later

Still failed

Retry again

10 minutes later

Most providers retry for
hours
or even days.

This is why

HTTP 200 is extremely important.

12. The Biggest Production Problem

Imagine

Stripe retries.

Your server receives
same webhook twice.

Payment Success

Payment Success

Oops.

What if your code is

```
wallet += 500;
```

First event

500

Second event

1000

Customer got double credit.

Huge bug.

13. Idempotency

One of the most important backend concepts.

Definition

An operation is idempotent
if executing it multiple times
produces the same final result.

Example

Bad

```
balance += 500;
```

Good

Transaction ID

Already processed?

↓

Yes

↓

Ignore

This is why almost every payment system stores

Event ID

Transaction ID

Payment ID

before processing.

Interview Question

Why is Idempotency important for Webhooks?

Expected Answer

Because webhook providers retry failed deliveries, duplicate events are possible. Idempotent processing prevents duplicate business operations like charging a customer twice.

14. Security Problem

Imagine

Someone sends

```
POST /webhook

{
  "payment": "success"
}
```

Directly to your server.

Will you trust it?

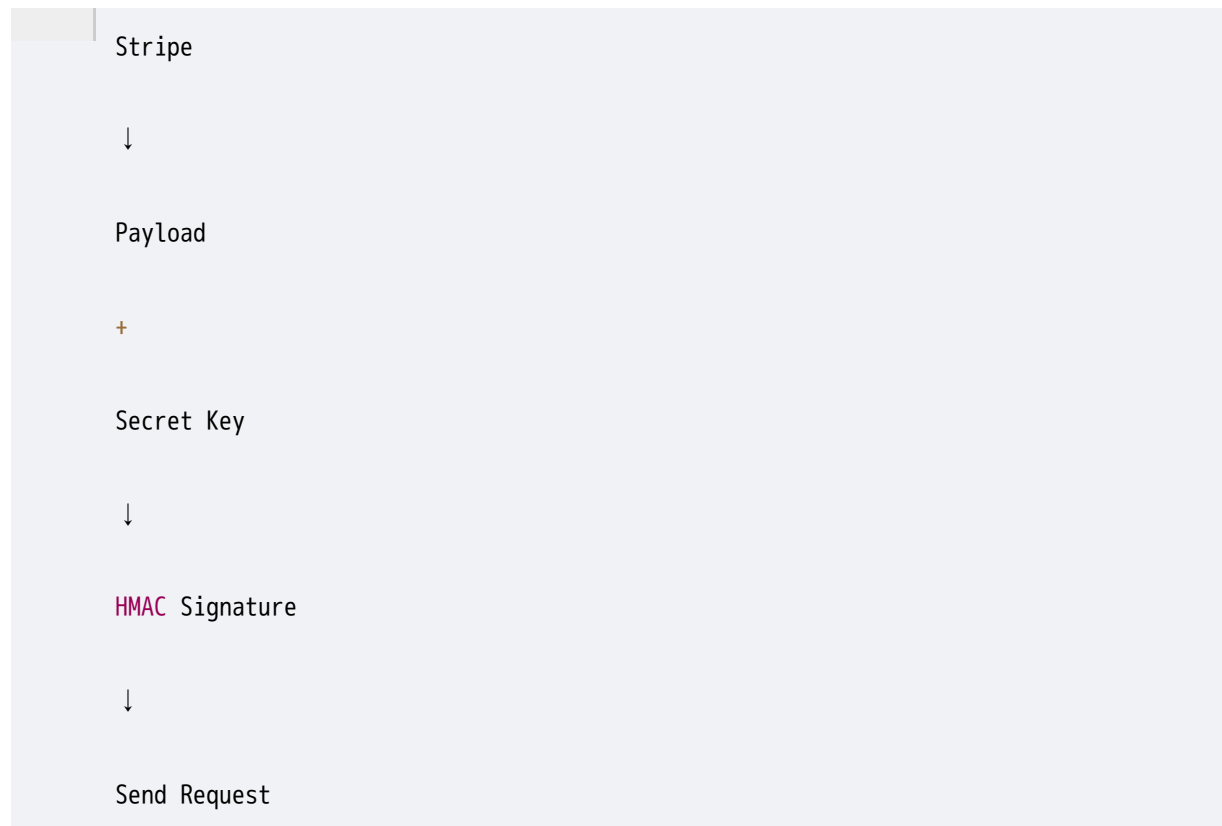
Obviously not.

15. Signature Verification

Professional providers

digitally sign every webhook.

Flow



Your server

computes

same HMAC.

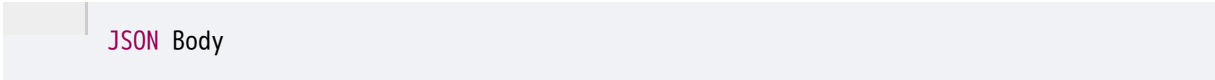
If both signatures match

Request is authentic.

Otherwise

Reject.

Never trust

A light blue rectangular box with a tab on the left labeled "JSON Body" in pink text.

alone.

Always verify signature.

16. Replay Attack

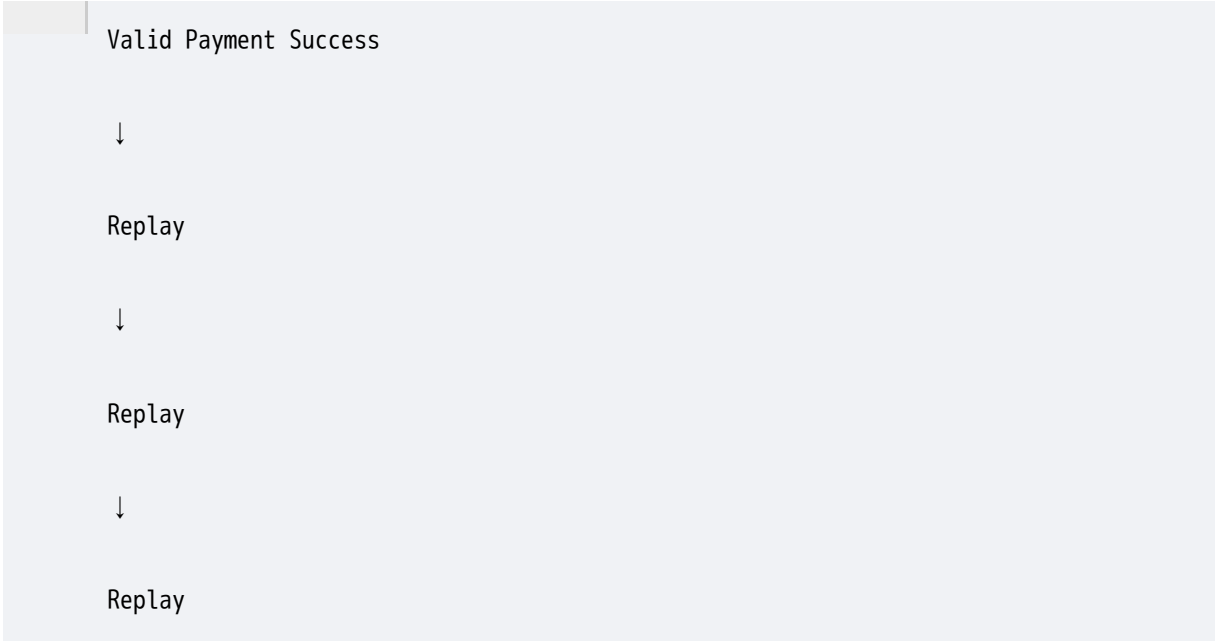
Another interview favorite.

Attacker records

a valid webhook.

Later

sends it again.

A light blue rectangular box containing a sequence of text and arrows. It starts with "Valid Payment Success", followed by a downward arrow, then "Replay", another downward arrow, "Replay", a third downward arrow, and finally "Replay".

Valid Payment Success

↓

Replay

↓

Replay

↓

Replay

Without protection

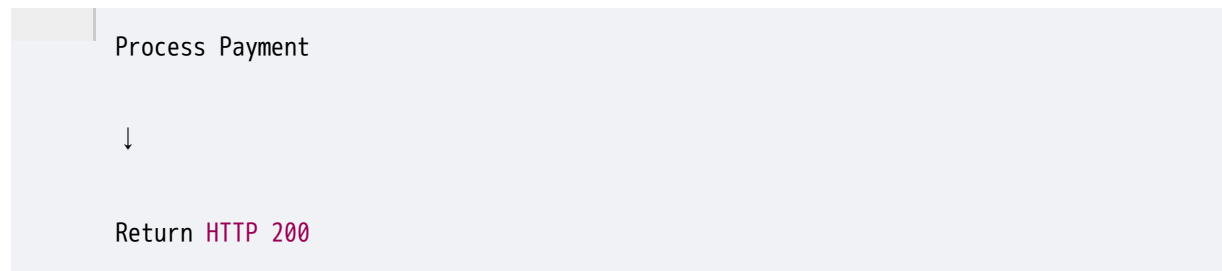
Customer may receive
multiple credits.

Solutions

- ✓ Timestamp
 - ✓ Signature
 - ✓ Event ID
 - ✓ Idempotency
-

17. Common Mistake

Many beginners write



Wrong.

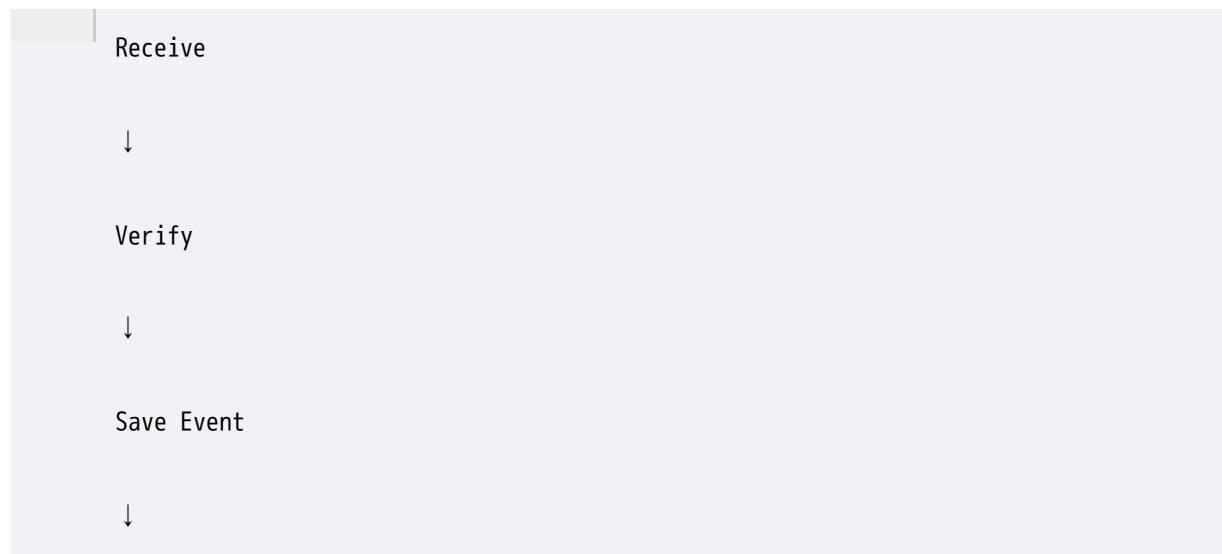
What if processing takes
30 seconds?

Webhook provider may timeout.

Retry.

Duplicate event.

Better



Return 200 Immediately



Background Worker

This pattern scales much better.

18. Polling vs Webhook

Polling	Webhook
Client asks repeatedly	Server notifies automatically
Many unnecessary requests	Only one request per event
Higher bandwidth	Lower bandwidth
Easier to implement	Slightly more complex
Higher latency	Near real-time

19. MAANG Interview Questions

Q1

What is a Webhook?

Best Answer

A Webhook is an event-driven HTTP callback where one server automatically sends an HTTP request to another server when a predefined event occurs.

Q2

Why not use Polling instead?

Because

Polling wastes

CPU

Bandwidth

Database queries

Network requests

Q3

Can Webhooks fail?

Yes.

Reasons

- Network failure
- Server down
- Timeout
- DNS issues

Therefore

Retries are mandatory.

Q4

Can duplicate Webhooks occur?

Absolutely.

Retries

Timeouts

Network failures

can all generate duplicate deliveries.

Q5

How do you prevent duplicate processing?

Expected Answer

Use

- Idempotency
 - Unique Event IDs
 - Transaction IDs
 - Database checks
-

Q6

How do you secure Webhooks?

Expected Answer

- HTTPS
 - HMAC Signature Verification
 - Timestamp Validation
 - Event IDs
 - Secret Keys
-

20. Senior Engineer Insight (4–10 YOE)

A junior engineer thinks:

"Webhook is just an HTTP POST."

A senior engineer immediately thinks about:

- retries
- duplicate deliveries
- idempotency
- replay attacks
- authentication
- timeout handling
- observability
- dead-letter queues for failed events

The HTTP POST is the easy part. Making it reliable in production is the real engineering challenge.